

Lume Telemetry Index — Build Spec

Oct 5, 2026 · @Tom

Purpose and goals

We are adding a bitmap-indexed, SQL-queryable telemetry index to Lume ("Lume TI"). It sits beside Signal K and signalk-parquet, so one query can combine numeric thresholds, discrete states, events, geography and full text across a fleet, and return in milliseconds. It applies Pilosa/FeatureBase ideas (roaring bitmaps, bit-sliced indexes, time-sharded column space) in Rust inside Lume. Apache DataFusion supplies the SQL engine.

The query it exists to answer:

```
SELECT vessel, date_bin(INTERVAL '10 minutes', ts) AS win,
max("environment.wind.speedTrue@max")
FROM telemetry
WHERE "propulsion.port.state" = 'started'
  AND "electrical.bilge.pumpCycles" > 3
  AND "environment.wind.speedTrue@max" > 12.9    -- 25 kn in m/s
  AND match(notes, 'leak OR water')
  AND ts > now() - INTERVAL '90 days'
GROUP BY vessel, win;
```

Goals

1. **G1 Full SQL over Signal K.** Full read-only SQL over bucketed telemetry and raw samples, for one vessel or a fleet: joins, CTEs, subqueries and window functions. Served over MCP, HTTP, CLI and the Postgres wire protocol, so psql, Grafana and DBeaver connect unchanged.
2. **G2 Fast selective queries.** Queries with four or more predicates run in bitmap space. See Benchmarks for targets on a Raspberry Pi 5 and on a shore host.
3. **G3 Text and telemetry in one column space.** `match()` over notes, logbook entries, notifications and Meridian VHF transcripts is just another bitmap filter.
4. **G4 Derived and rebuildable.** Raw truth stays in signalk-parquet and the delta stream. The index can be dropped and rebuilt at any time.
5. **G5 Agent-first.** Schema introspection, natural-language path resolution through Lume hybrid search, and an `explain` that says what was pushed down.

6. **G6 Fleet-native.** Each vessel indexes at the edge. Sealed shards replicate to shore over intermittent links, and a shore node queries every vessel.
7. **G7 Plugs into the boat's Pi computer.** One-click install on a Hat Labs HALPI2 running HaLOS (its container app store), and on any Raspberry Pi running OpenPlotter (the Signal K App Store). No compiler, no cloud, and no change to OpenCPN or the NMEA 2000 network. Pi 4 (4 GB) is the floor; the CM5/Pi 5 class is the target.

Non-goals for v1

- Replacing Signal K storage or the History API's provider. The History API adapter is a stretch goal.
- Raw-sample fidelity in the bitmap index. Raw samples are reached through a federated `raw` table that reads Parquet directly.
- Writes through SQL (no INSERT, UPDATE or DELETE).
- Clustering or consensus. Every node is single-process, and fleet scale comes from shard federation.
- Sub-second alerting. Rules and alarms stay in Signal K.
- Stored procedures, triggers or user-defined schemas. Queries get full read-only DataFusion SQL, and pushdown changes only speed, never which SQL is accepted.

Done means

- The golden query corpus (≥ 60 queries) returns results identical to the DuckDB oracle, within fixed-point tolerance.
- Benchmark targets are met on both reference machines.
- An agent connected only to Lume's MCP server can answer the motivating query from a plain-English question.
- On the pilot vessel's HALPI, Lume TI installs from the Signal K App Store in under 30 minutes. An energy query then runs from psql and from the plugin's webapp.

Pilot vessel (PV-1)

PV-1 is the v1 reference boat: the first real dataset, the first install, and the test of whether Lume TI really plugs into a Pi in an afternoon. She is a privately owned 16 m aluminium hybrid catamaran in fit-out, which replaces the owner's previous boat ("the previous boat" below). Her electronics were still being chosen as of February 2026, so ingest has to work with whatever ends up on the NMEA 2000 backbone.

PV-1 runs a **Hat Labs HALPI**. If it is the [HALPI2](#), it is a Raspberry Pi CM5 in an IP65 aluminium case. It has an NVMe SSD, an isolated CAN-FD port for NMEA 2000, an isolated RS-485 port for NMEA 0183, and runs from 10–32 V or from the NMEA 2000 bus. The HALPI2's default OS, [HaLOS](#), is container-based. Its Marine image ships Signal K, Grafana, InfluxDB and AvNav as containers behind Cockpit, a Homarr dashboard and single sign-on. A desktop image with Signal K and OpenCPN pre-installed is the alternative.

Three things follow for the build:

- **Ingest.** NMEA 2000 reaches Signal K straight from can0, so no gateway hardware is needed.
- **Backfill.** Existing history is in InfluxDB, not Parquet.
- **Install.** On HaLOS, the primary install path is a container app, not a process spawned from a Signal K plugin.

What we know about the boat

System	Equipment (status)	Reaches Signal K via	Main paths	TI fields
Propulsion	Nanni N4.80 diesel + 2 × 25 kW electric (published)	N2K engine gateway; format to confirm with Nanni	<code>propulsion.*.revolutions</code> , <code>.temperature</code> , <code>.state</code> , motor power	<code>bsi</code> , <code>set</code> , <code>count</code> (<code>@starts</code>)
Energy storage	40 kWh bank (published); Victron Multiplus 24 V on the previous boat	Victron VE.Direct or Venus OS via signalk-venus-plugin	<code>electrical.batteries.*.voltage</code> , <code>.current</code> , <code>.stateOfCharge</code>	<code>bsi</code>
Solar	3.75 kWp array (published); Victron SmartSolar MPPT on the previous boat	as above	<code>electrical.solar.*.panelPower</code> , yield	<code>bsi</code>

System	Equipment (status)	Reaches Signal K via	Main paths	TI fields
Navigation	B&G Zeus 3S, Halo 20+ radar, Triton 2 on the previous boat; our Oct 2025 research recommends B&G for 2.0 (not announced)	HALPI's native CAN port (can0), no extra gateway	<code>navigation.position</code> , <code>.speedOverGround</code> , <code>.courseOverGroundTrue</code> , <code>.headingMagnetic</code>	<code>bsi</code> , <code>geo</code>
Instruments	Maretron N2K wind, depth, speed, baro, temperature, humidity on the previous boat	N2K	<code>environment.wind.*</code> , <code>environment.depth.*</code> , <code>environment.outside.*</code>	<code>bsi</code>
Water	900 L tankage with roof rain catchment (published)	N2K tank senders	<code>tanks.freshWater.*.currentLevel</code>	<code>bsi</code> (slow profile)
AIS and VHF	AIS on the previous boat; Meridian VHF monitor proposed	N2K; Meridian publishes transcripts	AIS contexts; transcripts as Lume documents	<code>set</code> , <code>text</code>
Satellite link	Viasat Sailor 600 on the previous boat; Starlink unconfirmed	n/a	link state for <code>ti-sync</code>	—

Queries this boat makes worth it

She is an energy boat: 40 kWh, 3.75 kWp of solar, induction cooking on the previous boat, and hybrid drive. The first saved queries in the webapp should answer energy questions. Path names below are illustrative until the N2K list is final.

```
-- Daily energy balance, 10 s buckets: mean W × 10 s / 3.6e6 = kWh
SELECT date_trunc('day', ts) AS day,
       sum("electrical.solar.house.panelPower") * 10 / 3.6e6 AS solar_kwh,
       sum(coalesce("propulsion.port.motorPower", 0)
          + coalesce("propulsion.starboard.motorPower", 0)) * 10 / 3.6e6 AS
motor_kwh,
       min("electrical.batteries.house.stateOfCharge@min") AS min_soc
FROM telemetry
WHERE ts >= now() - INTERVAL '30 days'
GROUP BY day ORDER BY day;

-- Electric-only motoring: motors pulling, diesel off, at least 10 minutes
SELECT * FROM intervals(
  '"propulsion.port.motorPower" > 500 AND "propulsion.main.state" !=
  'started'',
  min_len => '10m');
```

Other questions an owner of this boat is likely to ask:

- How many miles did we do on sun alone this month?
- What did the rain catchment add?
- Which anchorages had VHF traffic mentioning weather while the wind was over 25 kn?

Assumptions to confirm with the owner

- ☐ Boat computer: a Hat Labs HALPI. Still to confirm: the exact model (Hat Labs documents the HALPI2; we found no "HALPI-M"), the CM5 RAM size, which OS image it runs (HaLOS Marine or the desktop image with OpenCPN), and whether OpenCPN really runs on the same HALPI as Signal K (believed so; see Single-box budget).
- ☐ The final NMEA 2000 equipment list: MFD brand, instrument vendor and engine gateway.
- ☐ Whether Victron stays on 2.0, and whether a Cerbo GX or Venus OS device is aboard.
- ☐ Consent to record 90 days of data for the benchmark dataset, and what of it may be shared publicly.
- ☐ Satellite link (Starlink or Viasat) and the data budget `ti-sync` may use.

Single-box budget

On PV-1, Signal K, OpenCPN and Lume TI are believed to share one HALPI. Navigation therefore always wins: Lume TI runs as a low-priority guest, and a heavy query can slow itself down but never OpenCPN or Signal K.

Process	Priority	Memory (planning estimate, 8 GB CM5)	CPU
OpenCPN	highest, untouched	0.5–1 GB	bursts on chart redraw
Signal K server	high, untouched	0.2–0.4 GB	light
InfluxDB + Grafana (HaLOS Marine only)	normal	0.3–0.6 GB	light
Lume TI ingest	low (<code>CPUWeight=50</code> , <code>nice 10</code>)	≤ 400 MB RSS	≤ 25 % of one core
Lume TI queries	lowest	1 GB pool (512 MB on 4 GB boards)	2 of 4 cores
Lume TI seal, backfill, sync	idle (<code>ionice -c3</code>)	within the ingest budget	paused under load

Rules enforced in `ti-serve` and the container or systemd unit:

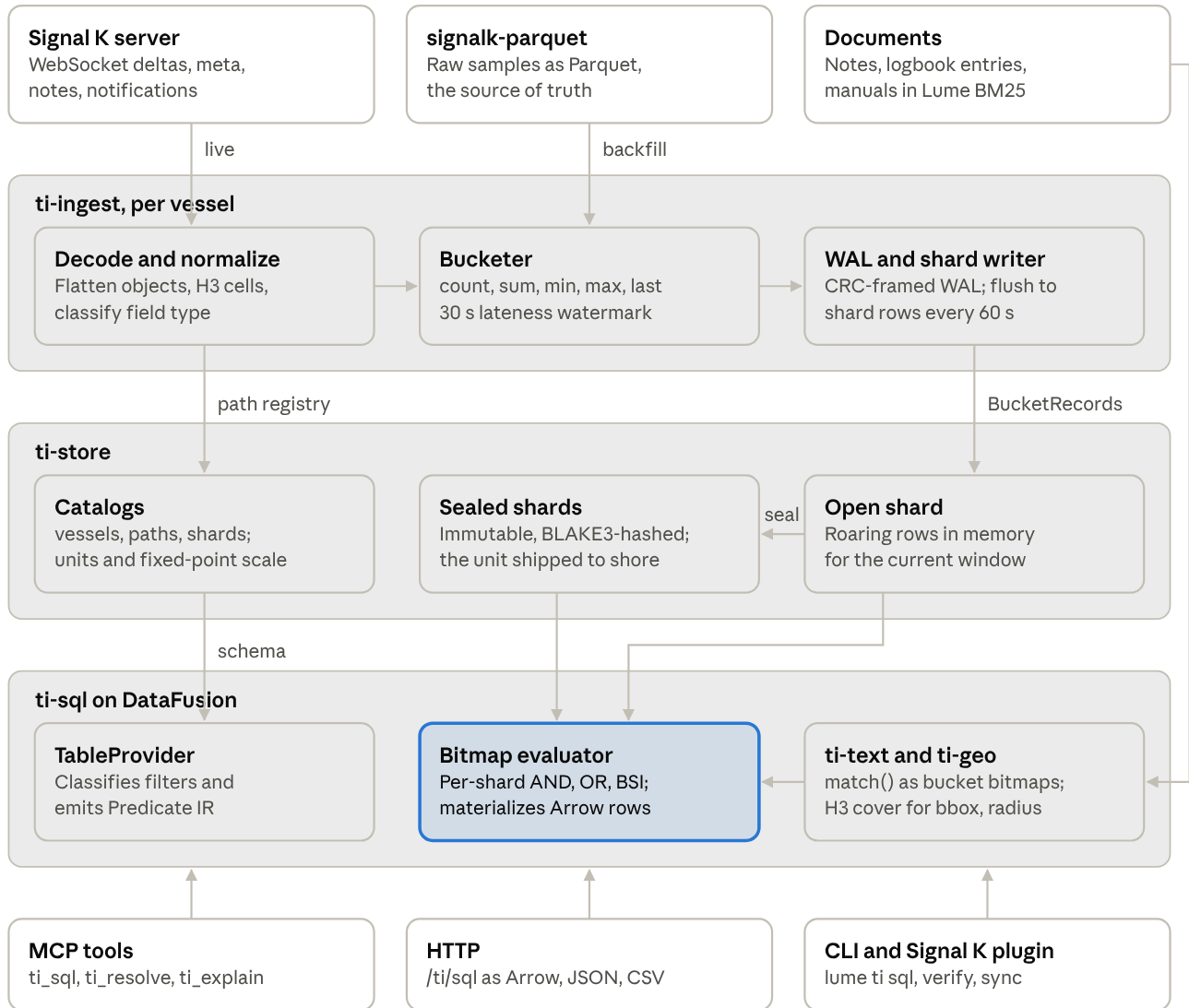
- **Hard cap.** `MemoryMax=1.5G` on the whole Lume TI unit. A query that needs more fails with a hint to narrow `ts` ; it never swaps the box.
- **Admission.** DataFusion runs with `target_partitions = 2` on the boat. One heavy query runs at a time, others queue, and the default timeout is 30 s.
- **Thermal.** The HALPI2 is passively cooled, and its host daemon reports SoC temperature. Above 75 °C, background jobs pause and query threads drop to 1.
- **Shutdown.** On the HALPI shutdown signal (power-button double-click or the host daemon), Lume TI flushes the WAL first. A power cut costs at most 60 s of buckets, which backfill can replay from InfluxDB or Parquet.
- **Contention test.** Gates M6 (see Benchmarks).

The memory figures are estimates to replace with measurements from PV-1's own box in M6.

System architecture

Lume TI is four layers in one Rust binary: ingest, store, query and interfaces. It runs on every vessel, and sealed shards are the only thing that moves to shore.

Telemetry flows down into bitmaps; queries return through one SQL engine



Lume TI architecture · 4 layers, one binary per vessel

The bitmap evaluator (highlighted) is the only place **Predicate** IR meets roaring rows, which is why it is the contract every lane builds against. Raw samples bypass the index entirely through the federated **raw** table over Parquet.

Data model

Every column is one time bucket for one vessel. Every field is a set of roaring-bitmap rows over those columns. A shard is one vessel $\times$ 65,536 consecutive buckets, so each row in a shard fits exactly one roaring container ($\leq$ 8 KB).

Column space

- **Bucket width W** : configurable per deployment, default 10 s. It is fixed once a store is created, and changing it means a rebuild.
- **Bucket index**: `b = floor((t - EPOCH) / W)`, with `EPOCH = 2020-01-01T00:00:00Z`.
- **Column id**: `col: u64 = (vessel_ord as u64) << 32 | b`. A 32-bit bucket index holds about 1,360 years at 10 s.
- **Shard key**: `(vessel_ord, b >> 16)`. At 10 s a shard spans 7.58 days, and a year is about 48 shards per vessel.
- **Local column**: `b & 0xFFFF`, stored in a `RoaringBitmap` (u32) per row per shard.
- **Vessel ordinal**: a dense u32 assigned by the `vessels` catalog and keyed by Signal K context URN (e.g. `vessels.urn:mrn:imo:mmsi:367000000`). The local `vessels.self` resolves to its URN at ingest.

Field types

Type	Used for	Rows per shard	Supported predicates
<code>presence</code>	One per indexed path; bit set if any sample landed in the bucket	1	<code>IS NOT NULL</code> , <code>IS NULL</code>
<code>set</code>	Strings, enums, booleans, <code>\$source</code> labels, derived states	1 per distinct value	<code>=</code> , <code>!=</code> , <code>IN</code> , <code>NOT IN</code>
<code>bsi</code>	Numeric values, one field per aggregate	exists + sign + <code>depth</code> magnitude rows	<code>=</code> , <code><</code> , <code><=</code> , <code>></code> , <code>>=</code> , <code>BETWEEN</code> ; <code>sum</code> , <code>min</code> , <code>max</code> , <code>count</code> pushdown
<code>count</code>	Event counts per bucket (pump cycles, alarms raised)	a <code>bsi</code> with no sign row	as <code>bsi</code>

Type	Used for	Rows per shard	Supported predicates
geo	H3 cells of <code>navigation.position</code> at resolutions 5, 7 and 9	1 per occupied cell	<code>in_bbox()</code> , <code>within_nm()</code> (inexact, refined by lat/lon BSI)
text	Lume BM25 documents mapped onto buckets	resolved on demand from Lume postings	<code>match()</code>

BSI encoding

- **Fixed-point:** `i = round(v × 10scale)`. Scale comes from the path's Signal K `meta.units` through the registry below, and a per-path override is allowed.
- **Sign-magnitude:** rows `exists`, `sign`, then `bit[0..depth)`. Depth grows when a value exceeds the current range. A wider depth only adds rows and never rewrites old ones.
- **Comparisons:** use the O'Neil/Quass bit-sliced range algorithm, as Pilosa did. It takes one pass over `depth` rows, AND/OR/ANDNOT only.
- **Tolerance:** the oracle comparison allows $\pm 0.5 \times 10^{\text{scale}}$.

Signal K unit	Scale	Resolution
rad	4	0.0001 rad ($\approx 0.006^\circ$)
m/s	3	1 mm/s
K	2	0.01 K
V	3	1 mV
A	2	10 mA
W	1	0.1 W
Pa	0	1 Pa
ratio	4	0.01 %
m	2	1 cm
Hz	1	0.1 Hz
s, J, C	0	1 unit

Signal K unit	Scale	Resolution
lat/lon (deg)	7	≈ 1.1 cm
unknown numeric	3	logged as a registry miss

Bucket aggregates

Each numeric path keeps running `count`, `sum`, `min`, `max` and `last` per bucket. The bucket closes into BSI fields named `path@agg`. The bare path name is an alias for `@mean`.

- **Default profile:** `@mean`, `@min`, `@max`.
- **slow profile:** `@last` only. Used for paths with a median sample interval $\geq W$, such as tank levels.
- **Opt-in:** `@last` and `@count` on any path.

Derived fields

These are computed at bucket close from rules in `ti.toml`:

- **Transitions:** e.g. `propulsion.*.state` changing to `started` becomes a `count` field `@starts`.
- **Edge counts:** a boolean path's rising edges per bucket, e.g. bilge pump cycles.
- **Notifications:** each `notifications.*` path becomes a `set` field over state values (`normal`, `alert`, `warn`, `alarm`, `emergency`), plus a `count` of raises.

Multiple sources

- The path's value comes from the preferred source, using Signal K source priorities when configured, otherwise the first source seen.
- Every source that reported in the bucket is recorded in a `path$source` set field, so `WHERE "navigation.speedOverGround$source" = 'n2k.115'` works.
- Per-source values are a v2 option.

Text mapping

Notes (Resources API), logbook entries and notification messages are indexed as Lume documents with `ts_start` and an optional `ts_end`. `match(notes, q)` runs Lume BM25 over the docs of that kind. Each hit becomes the bucket columns its time range covers. The resulting bitmap is cached per `(q, shard)` with an LRU.

Catalogs

These are stored as JSON in the store root and exposed as SQL tables:

- `vessels(ord, urn, name, mmsi, first_seen, last_seen)`
- `paths(path, field, agg, type, units, scale, depth, description, first_seen, last_seen)`, with description from Signal K `meta` and the spec
- `shards(vessel, shard_no, ts_from, ts_to, sealed, bytes, hash)`

Size budget

- **Worst case:** a dense BSI field at depth 16 is $18 \text{ rows} \times 8 \text{ KB} \times 48 \text{ shards} \approx 6.9 \text{ MB}$ per vessel-year at 10 s.
- **Typical boat:** 120 numeric paths under the default profile is about 360 fields, so $\leq 2.5 \text{ GB}$ per vessel-year worst case. Run containers and sparse paths bring the expected size to 0.6–1.0 GB.
- **Pi retention:** default 2 years. Older shards live only on shore.

Ingest pipeline

Live deltas and Parquet backfill feed the same bucketer. Buckets close on a watermark and land in shards through a write-ahead log, and a shard seals once it is entirely in the past.

Sources

1. **Live.** A Rust WebSocket client connects to `ws://<host>:3000/signal-k/v1/stream?subscribe=none`. It then sends `{"context":"vessels.self","subscribe":[{"path":"*","period":1000,"policy":"instant"}]}` and `{"context":"vessels.self","subscribe":[{"path":"notifications.*","policy":"instant"}]}`. Auth uses a Signal K device token from an access request, stored in `ti.toml`.
2. **Backfill.** Reads signal-k-parquet files with the arrow-rs `parquet` crate. It maps `received_timestamp / signal_k_timestamp`, `path`, `value` and `value_*` columns, and skips files whose content hash is already in the manifest. On HaLOS Marine, history lives in the bundled InfluxDB rather than in Parquet. A second backfill reader exports it by measurement and time range, and maps it to the same tuples.

3. **Meta.** On connect and every hour, it fetches `GET /signalk/v1/api/vessels/self` to read `meta` (units, displayName, description, zones) into the path registry.
4. **Documents.** It polls `GET /signalk/v2/api/resources/notes` every 60 s. It also takes logbook plugin entries when that plugin is installed, and notification messages from the live stream. Each becomes a Lume document with a time range.

Stages

1. **Decode.** Turn each delta into `(context, path, $source, timestamp, value)` tuples, using the update's `timestamp` and falling back to receive time if it's missing or skewed by more than 5 min.
2. **Normalize.** Flatten object values. `navigation.position` becomes `.latitude`, `.longitude` plus H3 cells, and `{value: {a, b}}` becomes `path.a`, `path.b`. Drop paths matching the deny-list (default: `*.ais.*` raw sentences and `design.*`).
3. **Classify.** Map numbers to `bsi`, and strings, booleans and enums to `set`. Objects are flattened and recursed. A path's first classification is sticky, and a later type change logs and creates a `path#2` field.
4. **Bucket.** Keep per-`(vessel, path)` accumulators keyed by bucket index: `count`, `sum`, `min`, `max`, `last`, sources seen, and edge or transition counters.
5. **Close.** A bucket closes when the watermark (max event time – 30 s lateness) passes its end. Closing emits `BucketRecord`s (see Fleet workstreams).
6. **Write.** Append to the per-vessel WAL (length-prefixed bincode with CRC32), then apply to in-memory shard rows. Flush dirty shard rows to disk every 60 s or every 50k records, then truncate the WAL to the flush point.
7. **Seal.** A shard seals once `now - shard_end > 1 h`. Sealing writes immutable per-field `.rbm` files (roaring portable format), computes a BLAKE3 hash over the field files, and updates the manifest.

Late and re-ingested data

- Data arriving after its bucket closed but before the shard seals re-opens that bucket for that field. This is a clear-and-rewrite, since BSI rewrite clears every row's bit for the column first.
- Late data after the shard sealed goes to a `repair` queue. A repair rewrites the shard into a new version and changes its hash. Shore picks the new version up by hash.
- Backfill always uses the clear-and-rewrite path, so it is idempotent.

Throughput targets

- **Pi 5:** $\geq 20,000$ values/s sustained live ingest, at $\leq 25\%$ of one core and ≤ 400 MB RSS.
- **Shore host, backfill:** $\geq 1,000,000$ values/s.

On-disk layout

```
ti/  
  ti.toml  
  catalog/vessels.json  catalog/paths.json  
  manifest.json          # shards, versions, hashes  
  wal/<vessel_ord>.wal  
  shards/<vessel_ord>/<shard_no>/v<version>/<field_id>.rbm  
  docs/                  # Lume index for notes, logs,  
  notifications
```

Query layer

DataFusion parses, plans and executes SQL. Lume TI is a `TableProvider` that takes pushed-down filters, evaluates them as bitmap operations per shard, and materializes only the surviving rows into Arrow. Pin one DataFusion major version per release, and upgrade only behind the golden corpus.

SQL surface

Table	Rows are	Key columns
telemetry	one per (vessel, bucket) with any data	vessel, ts (bucket start, UTC), one column per field: "path", "path@agg", "path\$source"; virtual notes, logbook, alerts for match()
docs	one per indexed document	vessel, kind, ts_start, ts_end, title, body, score (only populated with match())
raw	one per raw sample	DataFusion ListingTable over signalk-parquet files: exact samples, no bitmap acceleration

Table	Rows are	Key columns
vessels , paths , shards	catalog entries	as in Data model

Column names are exact Signal K paths and must be double-quoted. Agents never guess them: they call `ti_resolve` first (see Interfaces).

SQL coverage

The SQL is full read-only SQL: anything DataFusion accepts runs against every table here. Pushdown decides only how fast a query runs, never whether it runs. Supported:

- Joins of any kind across `telemetry` , `docs` , `raw` and the catalogs
- CTEs, including recursive ones
- Correlated and scalar subqueries
- Window functions (`lag` , `lead` , `row_number` , framed rolling aggregates)
- `UNION` , `INTERSECT` , `EXCEPT`
- `CASE` , `GROUP BY` / `HAVING` , `ORDER BY` / `LIMIT`
- DataFusion's scalar, aggregate and time functions (`date_bin` , `approx_percentile_cont` , and so on), plus the Lume TI functions below

INSERT, UPDATE, DELETE and DDL are rejected with an error that names the table as derived.

```
-- Window function over pushed-down rows: hours where house SOC fell faster
-- than 5 %/h
WITH h AS (
  SELECT date_bin(INTERVAL '1 hour', ts) AS hr,
         avg("electrical.batteries.house.stateOfCharge") AS soc
  FROM telemetry
  WHERE ts >= now() - INTERVAL '14 days'
  GROUP BY hr),
d AS (
  SELECT hr, soc, soc - lag(soc) OVER (ORDER BY hr) AS delta FROM h)
SELECT * FROM d WHERE delta < -0.05 ORDER BY hr;
```

The `ts` filter is pushed down. The CTE, the window and the outer filter run in DataFusion over the rows that survive it.

Functions

- `match(text_col, query) → bool`: Lume query syntax. Valid only in `WHERE` and always pushed down.
- `in_bbox(lat_min, lon_min, lat_max, lon_max) → bool`: covered by H3 cells, then refined against lat/lon BSI.
- `within_nm(lat, lon, radius_nm) → bool`: same approach, refined by haversine on the materialized rows.
- `date_bin`, `date_trunc` and every other function are standard DataFusion.
- `intervals(predicate_sql, min_len => '0s', max_gap => '0s', vessel => NULL)`: a table function. It returns `(vessel, start, end, buckets)` for each contiguous run of buckets where the predicate holds. It reads runs straight out of the result bitmap, using run containers when present, then merges gaps $\leq$ `max_gap`.

```
-- When did we motor in more than 25 kn of wind this season?
SELECT * FROM intervals(
  '"propulsion.port.state" = \'started\' AND "environment.wind.speedTrue@max"
> 12.9',
  min_len => '5m', max_gap => '1m')
WHERE start > '2026-05-01';
```

Pushdown

`supports_filters_pushdown` classifies each conjunct and returns:

Expression	Pushdown	Bitmap evaluation
<code>vessel = / IN</code>	Exact	prunes shards by vessel ordinal
<code>ts</code> comparisons and <code>BETWEEN</code>	Exact	prunes shards, then a range mask inside the boundary shards
<code>set_col = / != / IN / NOT IN</code>	Exact	row OR, or presence ANDNOT row
<code>bsi_col <op> literal</code> , <code>BETWEEN</code>	Exact	BSI range compare (literal scaled and rounded consistently)
<code>col IS [NOT] NULL</code>	Exact	presence row
<code>match(...)</code>	Exact	Lume postings mapped to buckets

Expression	Pushdown	Bitmap evaluation
<code>in_bbox</code> , <code>within_nm</code>	Inexact	H3 cover rows OR; DataFusion re-applies the exact predicate
<code>AND</code> , <code>OR</code> , <code>NOT</code> of the above	Exact	bitmap AND, OR, ANDNOT
Anything else (column-vs-column, arithmetic, UDFs)	Unsupported	DataFusion filters after materialization

Execution

1. **Plan.** Translate pushed filters into the `Predicate` IR (see Fleet workstreams). Prune to candidate shards using the vessel and ts bounds plus the manifest.
2. **Evaluate.** Each shard is a partition in DataFusion. Evaluating the IR yields a `RoaringBitmap` of local columns.
3. **Materialize.** Iterate the set bits in batches of 8,192. For each projected column, read values: `set` columns do a row-membership lookup per value, and `bsi` columns reconstruct the integer from depth rows and divide by 10^{scale} . Emit a `RecordBatch`.
4. **Stream.** Batches go to DataFusion operators (aggregate, sort, join) unchanged.

Aggregate rewrites (M4)

A physical optimizer rule replaces `Aggregate(Scan)` with a `BitmapAggregateExec` when every aggregate is one of the following, over pushed filters only:

- `count(*)`, computed as bitmap cardinality
- `count(col)`, computed as cardinality of filter AND presence
- `sum`, `min` or `max` of a `bsi` column, using the bit-sliced algorithms

The `GROUP BY` must be absent or limited to `vessel` and `date_bin(ts)`. Group windows become column-range masks. The target is $\geq 10\times$ faster than materialize-then-aggregate on year-long windows.

Explain

`EXPLAIN` and `ti_explain` print the DataFusion plan plus a Lume TI section. It lists shards pruned versus scanned, each conjunct's pushdown class, bitmap cardinality after each step, and the materialized row count. Agents use this to rewrite slow queries.

Interfaces

All surfaces are thin wrappers over one `TiEngine` handle. MCP is the primary consumer, because nemesis8 agents are the main users.

MCP tools (added to `lume serve`)

Tool	Input	Returns
<code>ti_resolve</code>	<code>phrase</code> , optional <code>vessel</code> , <code>limit</code> (default 8)	Ranked candidate columns with path, agg, units, description and last value. Uses Lume hybrid search over the <code>paths</code> catalog plus Signal K spec descriptions
<code>ti_schema</code>	optional <code>prefix</code> , <code>type</code>	Tables, columns, types, units, scale, time coverage
<code>ti_sql</code>	<code>sql</code> , <code>max_rows</code> (default 500), <code>format</code> (<code>json</code> , <code>csv</code> , <code>markdown</code>)	Rows, truncation flag, elapsed ms, and a one-line pushdown summary
<code>ti_intervals</code>	<code>predicate</code> , <code>min_len</code> , <code>max_gap</code> , time range	Interval list (sugar over the <code>intervals()</code> table function)
<code>ti_explain</code>	<code>sql</code>	Plan and pushdown report
<code>ti_status</code>	none	Ingest lag, WAL size, shards open or sealed, last sync per vessel

Tool output caps: 500 rows and 64 KB. Over that, the tool returns the first rows plus a hint to aggregate. Every result echoes the resolved units, so the agent never confuses m/s and kn.

HTTP (same port as `lume serve`)

- `POST /ti/sql` with body `{sql, format}`. Responds as Arrow IPC stream (`application/vnd.apache.arrow.stream`), JSON or CSV, picked from `Accept`.
- `GET /ti/schema`, `GET /ti/status`.
- `POST /ti/ingest` with NDJSON Signal K deltas, for sources that push instead of being subscribed to.

- `GET /ti/manifest`, `GET /ti/shards/{vessel}/{shard}/{version}.tar` for sync (see the fleet topology below).
- Auth reuses NUTS auth on shore. On the boat it binds to the LAN with an optional bearer token.

CLI (subcommands of `lume`)

```
lume ti init      --width 10s --root ./ti
lume ti ingest    --sk ws://openplotter.local:3000 --token $SK_TOKEN
lume ti backfill  --parquet ~/.signalk/data/parquet
lume ti sql       "SELECT ..." [--format table|csv|json]
lume ti seal      [--vessel self]          # force-seal past shards
lume ti verify    --oracle duckdb --corpus tests/golden
lume ti sync      --to https://shore.example/ti | s3://bucket/ti
lume ti serve     --port 8080              # alias for lume serve with TI
enabled
```

Signal K plugin `signalk-lume-ti` (thin, Node)

- **Packaging.** Ships as `signalk-lume-ti` in the Signal K App Store. Prebuilt `lume` binaries for linux-arm64 (Pi 4 and Pi 5 on 64-bit Raspberry Pi OS / OpenPlotter) and linux-x64 come as platform-specific optional npm dependencies. npm fetches the right one, and nothing compiles on the boat. Binaries are statically linked against musl, so the same file also runs inside a containerized Signal K. 32-bit armv7 is a stretch target.
- **Auth.** Handles the Signal K access-request flow and stores the device token, so the Rust process never needs a user password.
- **Supervision.** Runs `lume ti ingest` and `lume ti serve` as child processes and restarts them on crash. Shows ingest lag, WAL size and disk use in the Admin UI.
- **Webapp.** SQL console, saved queries with CSV download, and schema browser. The PV-1 energy queries ship as defaults.
- **Pin to chart.** On an explicit user action only, it writes query results to the Signal K Resources API, e.g. `intervals()` as notes or regions that Freeboard-SK displays. It writes only resources, never data paths.
- **Stretch (M6+).** Registers as a Signal K History API provider and answers `/signalk/v2/api/history/values` from Lume TI, so Freeboard-SK and KIP get fast history for free.

Postgres wire protocol

`lume ti serve --pg 5432` exposes the same DataFusion session over the Postgres wire protocol, using the `pgwire` crate. Any Postgres client works unchanged: `psql`, Grafana's PostgreSQL data source, DBeaver, pandas or `psycpg`.

- Simple and extended query protocols, read-only sessions.
- SCRAM password auth from `ti.toml`. Binds to the LAN interface only by default.
- DataFusion's `information_schema`, plus the minimal `pg_catalog` views Grafana and DBeaver need for table and column introspection.
- The memory pool is capped per query: 512 MB on Pi 4, 1 GB on Pi 5. Over the cap, the query errors with a hint to narrow `ts`.

Install on a HALPI2 (HaLOS)

On HaLOS, Lume TI ships as a container app in the HaLOS Marine store, next to Signal K, Grafana and InfluxDB. It is not a process spawned from inside the Signal K container.

- **Package.** `lume-ti-container`, built with Hat Labs' container-packaging-tools as an apt package. It goes upstream to `halos-marine-containers`, or into a DeepBlue store definition until it is accepted.
- **Image.** Multi-arch OCI image (arm64, amd64) holding the static `lume` binary. The store sits on a named volume on the HALPI's NVMe SSD.
- **Signal K.** Joins the Signal K container's network and subscribes over WebSocket. The token comes from a Signal K access request approved once in the Signal K admin UI. The thin Signal K plugin is optional here; it adds the webapp inside Signal K.
- **Routing.** HTTP and the webapp sit behind HaLOS's Traefik and Authelia single sign-on, with a tile on the Homarr dashboard. Postgres wire is published on LAN port 5432 with SCRAM auth.
- **Grafana.** The app provisions a PostgreSQL data source pointing at Lume TI, plus a starter energy dashboard built from the PV-1 saved queries.
- **Backfill.** The config form offers a one-time import from the bundled InfluxDB container, then live ingest takes over.

Steps:

1. Cockpit → Container Apps → Marine store → install Lume TI.
2. Approve its access request in Signal K.
3. Optionally run the InfluxDB backfill from the app's config form.

4. Open Lume TI from the Homarr dashboard, or query it from Grafana or psql.

Install on an OpenPlotter Pi

1. In Signal K Admin, open App Store, install `signalk-lume-ti` and restart.
2. Approve the plugin's access request under Security → Access Requests.
3. Pick a store path in Plugin Config. A USB or NVMe SSD is recommended, and the plugin warns if the store is on the SD card.
4. If signalk-parquet is installed, click Backfill to index existing history.
5. Open the webapp, or connect with `psql -h <pi> -p 5432 -U ti`.

Relationship to OpenCPN:

- OpenCPN keeps its existing Signal K or NMEA connection unchanged. Lume TI subscribes to Signal K separately and never transmits on NMEA 2000.
- In v1, OpenCPN shows no Lume TI results. Whether OpenCPN can display Signal K resources (notes, regions) is an open question. If it can't, a small OpenCPN plugin is a v2 option.

Fleet topology

- Each vessel runs `lume ti ingest` plus `serve`. Sealed shards are immutable and content-hashed, which makes them the unit of replication.
- `lume ti sync` diffs the local manifest against shore and ships missing shard versions. It resumes chunked uploads, so it tolerates Starlink and cellular drops.
- The open shard ships as WAL tail segments every 5 min when online, so shore lag stays at 5 min or less.
- The shore node mounts all vessels' shards. Its `telemetry` table spans every vessel ordinal, and vessel ordinals are re-mapped by URN at import.

Fleet workstreams

Nine crates, nine agent lanes. One lane (W0) freezes the shared contracts in week 1. After that, every lane builds against traits and fixtures and never waits on another lane's implementation. All crates live in the Lume workspace under `crates/ti-*`.

Lane	Crate	Owns	Depends on (contract only)	First deliverable
W0	ti-contracts	Types and traits below, ti.toml schema, golden corpus, synthetic generator	none	Merged contracts + 60 golden queries + DuckDB oracle results
W1	ti-core	Column addressing, set / bsi / presence / count rows, BSI range compare, sum, min, max, IR evaluator	W0	Predicate::eval passing proptests against a naive Vec<Option<i64>> model
W2	ti-store	WAL, shard files, manifest, seal, versioning, repair, mmap/read cache	W0	Crash-recovery test: kill -9 during flush, restart, no lost or duplicated record
W3	ti-ingest	SK WebSocket client, delta decode, normalize, classify, bucketer, watermark, unit/scale registry, Parquet backfill	W0	Replays a recorded delta log into BucketRecords identical to the oracle bucketing
W4	ti-sql	TableProvider, pushdown classifier, IR translation, materializer, intervals(), BitmapAggregateExec, explain	W0 (+ W1 via trait)	Golden corpus green against an in-memory ShardSource fixture
W5	ti-text	Doc ingestion from notes, logbook and notifications; match() → bucket bitmaps; LRU	W0, Lume search API	match() subset of the golden corpus green
W6	ti-geo	H3 indexing (h3o crate), bbox and radius cover, refine	W0	Cover never misses a point (proptest), ≤ 30 % false-positive buckets at res 9

Lane	Crate	Owns	Depends on (contract only)	First deliverable
W7	<code>ti-serve</code>	MCP tools, HTTP routes, Postgres wire, CLI, <code>ti_resolve</code> over Lume hybrid search, Signal K plugin	W0	Agent end-to-end: plain-English question → correct rows
W8	<code>ti-sync</code> + <code>ti-bench</code>	Manifest diff, resumable shard shipping, WAL-tail ship; benchmark harness and reports	W0, W2	Two-node sync test with a link that drops 20 % of chunks

Frozen contracts (W0, week 1)

Changing anything below requires a contracts PR approved by the integrator. Each lane mocks the others behind these.

```
pub type VesselOrd = u32;
pub type BucketIx  = u32;                // floor((t - EPOCH) / W)
pub type ColumnId  = u64;                // (vessel as u64) << 32 | bucket

#[derive(Clone, Copy, Hash, Eq, PartialEq, Ord, PartialOrd)]
pub struct ShardKey { pub vessel: VesselOrd, pub shard: u32 } // shard =
bucket >> 16

pub enum FieldKind { Presence, Set, Bsi { scale: u8 }, Count, Geo { res: u8 }
}

pub struct FieldSpec {
    pub id: u32, pub path: String, pub agg: Option<Agg>, pub kind: FieldKind,
    pub units: Option<String>,
}
pub enum Agg { Mean, Min, Max, Last, Count, Starts, Edges }

/// Output of the bucketer; input to the store.
pub struct BucketRecord {
    pub vessel: VesselOrd, pub bucket: BucketIx, pub field: u32,
    pub value: FieldValue, pub rewrite: bool,
```

```

}
pub enum FieldValue { Present, SetValue(u32 /* row id */), Int(i64),
Cells(Vec<u64>) }

/// The only language between SQL and bitmaps.
pub enum Predicate {
    All, None,
    Present(u32),
    SetEq { field: u32, rows: Vec<u32>, negate: bool },
    BsiCmp { field: u32, op: CmpOp, lo: i64, hi: Option<i64> }, // hi for
BETWEEN
    TsRange { from: BucketIx, to: BucketIx }, // inclusive
    Text { kind: String, query: String },
    GeoCover { field: u32, cells: Vec<u64> },
    And(Vec<Predicate>), Or(Vec<Predicate>), Not(Box<Predicate>),
}
pub enum CmpOp { Eq, Ne, Lt, Le, Gt, Ge, Between }

pub trait ShardSource: Send + Sync {
    fn shards(&self, vessels: Option<&[VesselOrd]>, from: BucketIx, to:
BucketIx) -> Vec<ShardKey>;
    fn eval(&self, shard: ShardKey, p: &Predicate) -> Result<RoaringBitmap>;
    // local cols
    fn read(&self, shard: ShardKey, cols: &RoaringBitmap, fields: &[u32]) ->
Result<RecordBatch>;
    fn agg(&self, shard: ShardKey, cols: &RoaringBitmap, field: u32, a:
AggOp) -> Result<AggPartial>;
}
pub trait ShardSink: Send {
    fn apply(&mut self, recs: &[BucketRecord]) -> Result<()>;
    fn flush(&mut self) -> Result<()>;
    fn seal(&mut self, key: ShardKey) -> Result<ShardManifestEntry>;
}
pub trait TextIndex: Send + Sync {
    fn match_buckets(&self, vessel: VesselOrd, kind: &str, q: &str, from:
BucketIx, to: BucketIx)
        -> Result<RoaringTreemap>; // global ColumnIds
}

```

Rules for every agent PR

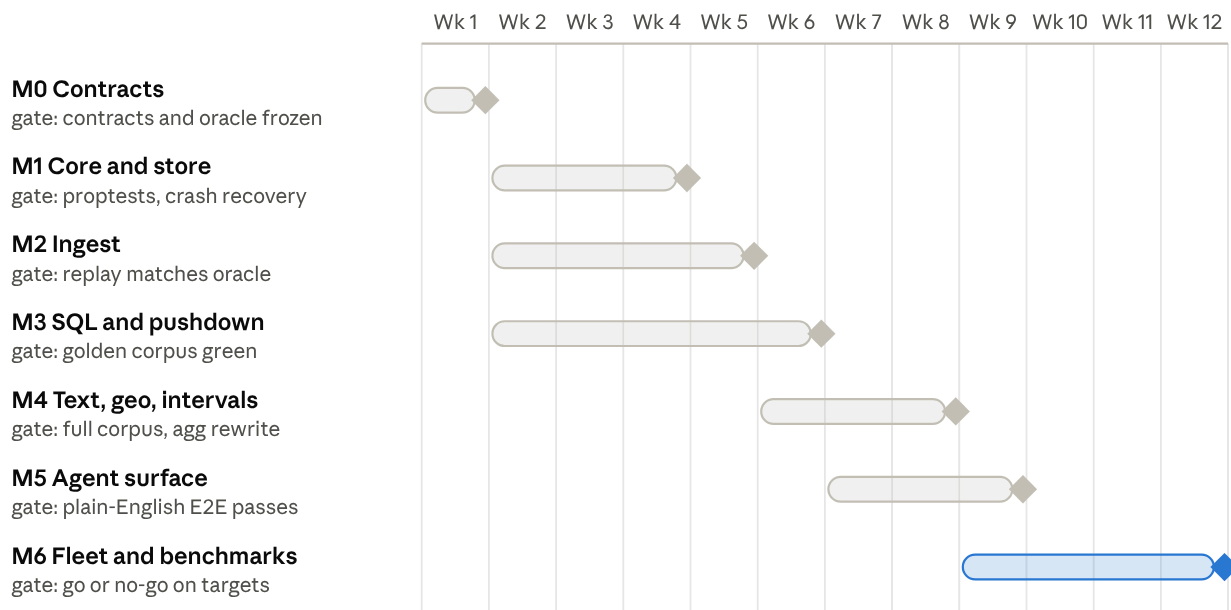
- `cargo test`, `cargo clippy -- -D warnings`, and `cargo fmt --check` all pass. No `unsafe` outside `ti-store` mmap code, and that code needs a comment justifying it.
- Every new predicate or aggregate path adds at least one golden query and one proptest.
- Performance-sensitive changes attach a `ti-bench` before/after table to the PR.
- No new runtime dependency without a line in the decisions log. C bindings are allowed only for `croaring`, and only after the M4 evaluation.
- Lanes integrate on `main` daily behind a feature flag (`ti`). The integrator agent runs the full corpus nightly and files regressions back to the owning lane.

Milestones and acceptance

The plan runs 12 weeks from kickoff. W0 freezes contracts in week 1, then core, ingest and SQL build in parallel from week 2 against fixtures. The week counts are a proposal, sized for one agent per lane plus an integrator.

Three lanes start in week 2; the go or no-go lands at week 12

Weeks from kickoff. Bar = build window, diamond = acceptance gate.



Proposed 12-week plan · 7 milestones, each closed by a gate

A milestone closes only when every gate test below passes in CI on `main`.

M0 Contracts (W0)

- ☐ `ti-contracts` merged with the types and traits from Fleet workstreams, plus doc comments
- ☐ `ti.toml` schema with defaults ($W = 10$ s, profiles, allow/deny lists, unit-to-scale table)
- ☐ `ti-bench gen` produces the 5-vessel $\times$ 90-day correctness set deterministically from a seed
- ☐ ≥ 60 golden queries covering Q1-Q8, each with a DuckDB oracle twin and stored expected output

M1 Core and store (W1, W2)

- ☐ BSI compare, sum, min and max agree with a naive model on 10,000 proptest cases, including negatives and depth growth
- ☐ `Predicate::eval` agrees with the naive model for random AND/OR/NOT trees of depth ≤ 4
- ☐ Crash test: 1,000 kill -9 runs during flush, with zero lost or duplicated records after replay
- ☐ Seal produces byte-identical files and hashes from identical input

M2 Ingest (W3)

- ☐ Replaying a recorded 24 h delta log yields `BucketRecord`s equal to oracle bucketing
- ☐ Parquet backfill of the correctness set is idempotent: running it twice gives identical manifest hashes
- ☐ Pi 5 sustains 20,000 values/s for 1 h within the CPU and RSS budget

M3 SQL and pushdown (W4)

- ☐ All non-text, non-geo golden queries match the oracle, first on the in-memory fixture, then on a real store
- ☐ `EXPLAIN` shows Exact pushdown for every expression the pushdown table marks Exact
- ☐ `raw` table queries the same Parquet and matches DuckDB exactly

M4 Text, geo, intervals (W4, W5, W6)

- ☐ Full golden corpus green, including `match()`, `in_bbox`, `within_nm` and `intervals()`

- ☐ `BitmapAggregateExec` is $\geq 10\times$ faster than the materializing path on Q4 at shore scale
- ☐ `croaring` frozen-view evaluation written up in the decisions log, adopt or reject

M5 Agent surface (W7)

- ☐ MCP tools are live in `lume serve`, and `ti_resolve` returns the right column in the top 3 for $\geq 90\%$ of a 100-phrase test set
- ☐ A nemesis8 agent with only Lume MCP answers 20 scripted fleet questions, and the integrator grades the answers against oracle results
- ☐ The Signal K plugin installs on a HALPI2 from the HaLOS Marine container store, and on OpenPlotter from the Signal K App Store (Pi 4 4 GB and Pi 5), obtains a token and supervises ingest. psql and Grafana connect over Postgres wire and pass a 20-query smoke set

M6 Fleet and benchmarks (W8, integrator)

- ☐ Two-node sync converges with 20 % chunk loss and a 30-minute link outage
- ☐ The shore node answers fleet queries across 50 synthetic vessels
- ☐ Benchmark report against every target. Go or no-go is recorded in the decisions log

Benchmarks and evaluation

Correctness is defined by a DuckDB oracle over the same Parquet. Speed is judged against that same DuckDB baseline on two reference machines. The targets below are proposals to confirm at M0, not measurements.

Reference machines

- **Edge:** Hat Labs HALPI2 (CM5, 8 GB, built-in NVMe), the PV-1 computer, 1 vessel. Same SoC class as a Pi 5. Floor: Pi 4, 4 GB, USB SSD, which must pass the golden corpus with p95 targets relaxed 3 $\times$.
- **Shore:** 8-core x86-64, 64 GB RAM, NVMe, 50 vessels.

Datasets

1. **Synthetic fleet (`ti-bench gen`).** 50 vessels $\times$ 365 days $\times$ 120 paths (10 navigation paths at 1 Hz, the rest at 0.1 Hz), about 33 billion raw values, written in the signalk-parquet layout. The full set is for performance runs only. Correctness runs use a 5-

vessel × 90-day subset. The generator models passages, anchoring and dock time. It includes engine on/off states, bilge cycles correlated with heel, wind fronts, notifications, and notes with planted keywords. The seed is fixed so results are reproducible.

2. **Real boat.** At least 90 days of recorded deltas from PV-1 (see Pilot vessel), or another DeepBlue test vessel if her install slips. This catches path sprawl, source conflicts and timestamp skew that synthetic data hides.

Oracle

The oracle uses DuckDB SQL over the same Parquet. Each golden query has an oracle twin that buckets with `time_bucket(INTERVAL 'W', ts, TIMESTAMP '2020-01-01')`. It then applies the same aggregate and fixed-point rounding.

`lume ti verify` runs both and diffs the results:

- Exact on keys, set values and counts.
- $\pm 0.5 \times 10^{\wedge}$ -scale on BSI values.

Query classes and targets

Class	Example	Edge p95 (1 vessel, 1 yr)	Shore p95 (50 vessels, 1 yr)
Q1 point lookup	wind at a timestamp	≤ 20 ms	≤ 20 ms
Q2 selective multi-predicate (4+ conjuncts, < 1 % selectivity)	motivating query	≤ 150 ms	≤ 100 ms
Q3 count with filters	buckets with SOG > 6 kn and engine off	≤ 50 ms	≤ 60 ms
Q4 windowed aggregate	max wind per day, 1 year	≤ 400 ms	≤ 300 ms
Q5 intervals	motoring in > 25 kn, runs ≥ 5 min	≤ 150 ms	≤ 150 ms
Q6 text + telemetry	<code>match(notes, 'leak')</code> and heel > 20°	≤ 200 ms	≤ 150 ms
Q7 geo + telemetry	within 5 nm of a point, depth < 3 m	≤ 300 ms	≤ 250 ms

Class	Example	Edge p95 (1 vessel, 1 yr)	Shore p95 (50 vessels, 1 yr)
Q8 broad scan (selectivity > 30 %)	mean battery voltage per hour	DuckDB parity ± 50 %	DuckDB parity ± 50 %

Q2, Q3, Q5 and Q6 must also beat the DuckDB baseline by at least 5× on shore. If they don't, the decisions log records why bitmaps aren't paying off, and the fallback in Risks is triggered.

Single-box contention test (gates M6). On the HALPI2, run OpenCPN with continuous chart pan and zoom while Signal K ingests at full rate. Then run Q4 and Q8 back to back for 10 minutes. The test passes if:

- Signal K delta latency (sensor timestamp to WebSocket client) rises by less than 10 % against the same run without Lume TI.
- No delta is dropped.
- OpenCPN shows no visible stutter, measured as 95th-percentile frame time.

Reported per run

- p50, p95 and p99 per class, plus cold and warm cache timings.
- Index bytes per vessel-year, compared with the Parquet bytes.
- Ingest values/s, CPU % and RSS on the Pi.
- Sync bytes per day per vessel.
- Results go to `bench/results/<date>-<git sha>.json` with a markdown summary, and CI fails on a > 15 % p95 regression.

Risks, open questions and decisions

The biggest risk is that bitmaps don't beat DuckDB by enough on single-boat data to justify a second engine. The M6 benchmark gate exists to find that out early and cheaply.

Risks

Risk	Signal	Mitigation
Bitmap speedup < 5× vs DuckDB on Q2, Q3, Q5, Q6	M6 bench report	Keep <code>ti-sql</code> and route broad queries to the <code>raw</code> table. Ship only text, geo and <code>intervals()</code> , where bitmaps are unique
Pi memory pressure on year-long queries	RSS > 1 GB in Q4	Shard-at-a-time streaming, mmap read path, evaluate <code>croaring</code> frozen views in M4
Path explosion (AIS, per-device sources, plugin noise)	> 2,000 fields per vessel	Allow/deny lists in <code>ti.toml</code> , <code>slow</code> profile auto-detection, field cap with a <code>ti_status</code> warning
Fixed-point surprises (wrong units, missing meta)	Registry misses logged at ingest	Unit-to-scale table plus per-path override. <code>ti_status</code> lists paths without units
Clock skew and backfilled history	Updates far from receive time	5-min skew rule, repair queue, versioned shards
DataFusion API churn	Breaking change on upgrade	Pin the major version and upgrade quarterly behind the golden corpus
Bucket semantics mislead users ("max" of a 10 s mean)	Agent answers disagree with raw	<code>@agg</code> naming is mandatory in output, <code>ti-sql</code> echoes it, and <code>raw</code> is available for exact checks

Open questions

- ☐ Default bucket width: 10 s, or 1 s with 1-min rollups? (1 s is 10× the index size)
- ☐ Per-source values as first-class columns in v1, or only `$source` sets?
- ☐ AIS contacts as a second table (`contacts` , columns = observer × bucket, rows = MMSI), or out of scope?
- ☐ Shore storage: local NVMe only, or sealed shards in object storage with a read cache?
- ☐ Should `ti_resolve` also index Signal K spec descriptions for paths a vessel has never reported?

- ☐ Licensing check: borrow algorithm ideas only from FeatureBase (Apache-2.0), and copy no code, to keep Lume BSD-3-clean.

Decisions log

#	Decision	Why
D1	The index is derived and rebuildable. Parquet and deltas stay the source of truth	No migration risk, and the oracle comes for free
D2	Build it in Lume (Rust, roaring) rather than revive FeatureBase (Go, archived Feb 2024)	One binary, BM25 postings already roaring-native, Pi-friendly
D3	DataFusion for SQL	Mature planner, Arrow-native, <code>TableProvider</code> pushdown API, can also query Parquet (<code>raw</code>)
D4	Shard = vessel × 2 ¹⁶ buckets	One roaring container per row per shard. Vessel-local seal and sync
D5	The sealed, content-hashed shard is the unit of replication	Idempotent sync over flaky links. Shore needs no merge logic
D6	The DuckDB oracle defines correctness	Deterministic acceptance for agent lanes
D7	<code>Predicate</code> IR is the only interface between SQL and bitmaps	Lets W1 and W4 build in parallel from day one

Sources

- [Lume README](#): hybrid search, roaring-bitmap SKG, MCP server
- [FeatureBase repository](#): archived Feb 21, 2024, Apache-2.0
- [signalk-parquet README](#): Parquet archive, DuckDB querying, History API provider
- [Signal K History API](#): `/signalk/v2/api/history` , from / to / duration parameters
- [Signal K Resources API, provider methods](#): filtering is left to provider plugins
- [HALPI2 documentation](#): CM5 boat computer, NVMe, CAN-FD NMEA 2000, RS-485 NMEA 0183

- [HALPI2 operating system images](#): HaLOS container distribution, Marine image with Signal K, Grafana, InfluxDB, AvNav
- [halos-marine-containers](#): HaLOS Marine container store definitions